Real-Time Wildfire Monitoring Daemon with Temporal Differentiation

The Physics of Temporal Differentiation ($\frac{\partial T}{\partial t}$)

A single satellite snapshot evaluates spatial contrast (how hot a pixel is compared to its immediate neighbors). However, the most reliable physical discriminator between solar heating and nascent combustion is **temporal rate-of-rise**:

```
DIURNAL HEATING vs. NASCENT COMBUSTION
```

1. `Diurnal Solar Ground Heating (Thermal Inertia of Rock & Soil):`
   - `Solar radiation warms the terrain gradually over hours.`
   - `Maximum rate of rise: (T)/t  +0.1 K to +0.3 K per 5-minute interval.`

2. `Nascent Vegetation Combustion (Discontinuous Exothermic Ignition):`
   - `An ignition transforms dry brush into an 800 K{1,200 K flaming front in minute`
   - `Sub-pixel midwave radiance spikes discontinuously:`
     `(T)/t  +1.5 K to +6.0 K per 5-minute interval!`

By computing the temporal difference between the current frame (t) and the previous frame ($t-1$):

$$\Delta T_{\text{temporal}}(y, x) = \Delta T_t(y, x) - \Delta T_{t-1}(y, x)$$

The system achieves three operational objectives:

- **Suppression of Static Thermal Anomalies:** Quarries, solar farms, and bare rocky peaks that remain consistently warm have $\Delta T_{\text{temporal}} \approx 0.0$ K and are suppressed.
- **Early Detection of Nascent Ignitions:** A pixel exhibiting an abrupt rise ($\Delta T_{\text{temporal}} \geq +1.5$ K) alongside elevated model confidence ($\hat{p} \geq 0.70$) triggers a high-priority NASCENT_IGNITION alert before reaching the strict static threshold ($\tau = 0.90$–0.95).
- **Fire Front Vector Tracking:** By calculating the centroid displacement between $t-1$ and t, the system computes fire propagation speed and heading.

Production Script: live_fire_monitor.py

This script provides an operational monitoring daemon. It supports two execution modes:

- **SIMULATION Mode** (Default for testing): Sequences through consecutive scenes from your processed_800 test split at 5-minute intervals to verify temporal differentiation and alert dispatching immediately.
- **LIVE Mode:** Connects to the EUMETSAT Data Store API via eumdac, polls every 5 minutes for new Meteosat RSS Level 1.5 products, processes them in memory, and dispatches real-time incident reports.

Create live_fire_monitor.py in D:\ramy uni\AI ENGENEERING\FINDFIRE\findfire\:

```
import os
import sys
import time
import json
import tempfile
import zipfile
from datetime import datetime, timedelta
import numpy as np
import pandas as pd
from scipy.ndimage import label, center_of_mass
import matplotlib.pyplot as plt
import torch
import torch.nn.functional as F

from unet_model import UNet

#
    ================================================================

# 1. OPERATIONAL ENGINE CONFIGURATION
#
    ================================================================

# Execution Mode: "SIMULATION" (test on consecutive historical
    scenes) or "LIVE" (poll EUMETSAT API)
MONITORING_MODE = "SIMULATION"

# EUMETSAT API Credentials (Required if MONITORING_MODE = "LIVE")
CONSUMER_KEY = "YOUR_CONSUMER_KEY_HERE"
CONSUMER_SECRET = "YOUR_CONSUMER_SECRET_HERE"

# Model Checkpoints (Case 1 & Case 2 800-scene models)
NIGHT_CKPT = os.path.join("checkpoints", "best_unet_night_800.pth
    ")
DAY_CKPT = os.path.join("checkpoints", "best_unet_day_800.pth")
INDEX_PATH = r"D:\FINDFIRE_DATA\processed_800\dataset_index_800.
    csv"
```

```
# Output Directories
ALERT_DIR = "live_alerts"
ALERT_LOG_FILE = os.path.join(ALERT_DIR, "active_incidents.jsonl"
    )
os.makedirs(ALERT_DIR, exist_ok=True)

# Geographic Grid for Northern Algeria
LAT_MIN, LAT_MAX = 34.0, 37.5
LON_MIN, LON_MAX = -2.0, 8.5
GRID_H, GRID_W = 120, 250
PIXEL_AREA_KM2 = 9.0  # ~3 km x 3 km nominal resolution

# Static Land Mask (Discard Mediterranean Sea above 36.9 N )
lat_per_row = np.linspace(LAT_MAX, LAT_MIN, GRID_H)
LAND_MASK = np.repeat((lat_per_row <= 36.9)[:, None], GRID_W,
    axis=1)

# Detection Thresholds
TAU_DAY = 0.90
TAU_NIGHT = 0.95
TAU_NASCENT_DAY = 0.70    # Relaxed threshold if high temporal
    rise is detected
TAU_NASCENT_NIGHT = 0.75
RATE_OF_RISE_THRESHOLD_K = 1.5  # +1.5 Kelvin jump in 5 minutes
    indicates combustion

# Normalization Constants
NIGHT_MEANS = torch.tensor([295.0, 285.0, 10.0]).view(3, 1, 1)
NIGHT_STDS = torch.tensor([15.0, 12.0, 8.0]).view(3, 1, 1)
DAY_MEANS = torch.tensor([315.0, 300.0, 15.0, 0.0]).view(4, 1, 1)
DAY_STDS = torch.tensor([15.0, 12.0, 8.0, 4.0]).view(4, 1, 1)

device = torch.device("cuda" if torch.cuda.is_available() else "
    cpu")


#
    ================================================================


# 2. MODEL INITIALIZATION
#
    ================================================================

print("=" * 75)
print("FINDFIRE: REAL-TIME OPERATIONAL WILDFIRE DAEMON")
print(f"Hardware Engine: {device} | Mode: {MONITORING_MODE}")
print("=" * 75)

if not os.path.exists(NIGHT_CKPT) or not os.path.exists(DAY_CKPT)
    :
     sys.exit("Error: Production checkpoints not found. Run
```

```
        training before launching live daemon.")

print("Loading UNet_Night (3 Channels)...")
night_state = torch.load(NIGHT_CKPT, map_location=device,
    weights_only=False)
model_night = UNet(in_channels=3, out_channels=1, base_features
    =64).to(device)
model_night.load_state_dict(night_state["model_state_dict"])
model_night.eval()

print("Loading UNet_Day (4 Channels)...")
day_state = torch.load(DAY_CKPT, map_location=device,
    weights_only=False)
model_day = UNet(in_channels=4, out_channels=1, base_features=64)
    .to(device)
model_day.load_state_dict(day_state["model_state_dict"])
model_day.eval()
print("Neural models loaded successfully.\n")


#
    ================================================================

# 3. SPATIAL & INFERENCE CORE
#
    ================================================================

def run_spatial_inference(raw_3ch, dt_timestamp):
    """
    Executes neural forward pass with regime routing and spatial
        contrast.
    """
    hour = dt_timestamp.hour
    is_night = (hour < 6) or (hour > 18)
    regime = "NIGHT" if is_night else "DAY"
    img_tensor = torch.from_numpy(raw_3ch).float()

    if is_night:
        norm_img = (img_tensor - NIGHT_MEANS) / NIGHT_STDS
        padded = F.pad(norm_img, (3, 3, 4, 4), mode="reflect").
            unsqueeze(0).to(device)
        with torch.no_grad():
            logits = model_night(padded)
            probs = torch.sigmoid(logits)[0, 0, 4:124, 3:253].cpu
                ().numpy()

        # Apply physical ground temperature constraint
        t_39 = raw_3ch[0]
        probs = probs * (t_39 >= 285.0).astype(np.float32)
    else:
        # Compute 11x11 Local Background Contrast
```

```
        delta_t = img_tensor[2:3, :, :].unsqueeze(0)
        local_bg = F.avg_pool2d(delta_t, kernel_size=11, stride
            =1, padding=5).squeeze(0)
        contrast = img_tensor[2:3, :, :] - local_bg
        img_4ch = torch.cat([img_tensor, contrast], dim=0)

        norm_img = (img_4ch - DAY_MEANS) / DAY_STDS
        padded = F.pad(norm_img, (3, 3, 4, 4), mode="reflect").
            unsqueeze(0).to(device)
        with torch.no_grad():
            logits = model_day(padded)
            probs = torch.sigmoid(logits)[0, 0, 4:124, 3:253].cpu
                ().numpy()

    # Enforce static land barrier
    probs = probs * LAND_MASK
    return probs, regime


#
    ================================================================

# 4. TEMPORAL RATE-OF-RISE TRACKER
#
    ================================================================

class TemporalFireTracker:
    """
    Maintains chronological frame buffer to compute temporal
        differentials.
    """
    def __init__(self):
        self.prev_timestamp = None
        self.prev_delta_t = None
        self.prev_probs = None

    def analyze_frame(self, raw_3ch, current_timestamp):
        current_delta_t = raw_3ch[2]  # Channel 2: Delta T = T3.9
            - T10.8
        current_probs, regime = run_spatial_inference(raw_3ch,
            current_timestamp)

        tau_standard = TAU_NIGHT if regime == "NIGHT" else
            TAU_DAY
        tau_nascent = TAU_NASCENT_NIGHT if regime == "NIGHT" else
            TAU_NASCENT_DAY

        # Default state when no t-1 frame is cached (first boot
            or gap)
        rate_of_rise = np.zeros_like(current_delta_t)
        time_delta_mins = 5.0
```

```
        if self.prev_delta_t is not None and self.prev_timestamp
           is not None:
            time_delta_mins = (current_timestamp - self.
               prev_timestamp).total_seconds() / 60.0

            # Compute temporal derivative only if interval is
               within operational bounds (3 to 15 min)
            if 3.0 <= time_delta_mins <= 15.0:
                dt_factor = 5.0 / time_delta_mins
                # Normalized rate of rise in Kelvin per 5 minutes
                rate_of_rise = (current_delta_t - self.
                   prev_delta_t) * dt_factor
                rate_of_rise = np.clip(rate_of_rise, -20.0, 50.0)

        # ---------------- HYBRID DECISION LOGIC ----------------
        # Condition A: Standard high-confidence combustion
           detection
        standard_mask = (current_probs >= tau_standard)

        # Condition B: Incipient nascent ignition (rapid thermal
           jump + elevated confidence)
        nascent_mask = (current_probs >= tau_nascent) & (
           rate_of_rise >= RATE_OF_RISE_THRESHOLD_K)

        combined_mask = (standard_mask | nascent_mask) &
           LAND_MASK

        # Update frame buffer
        self.prev_timestamp = current_timestamp
        self.prev_delta_t = current_delta_t.copy()
        self.prev_probs = current_probs.copy()

        return {
            "combined_mask": combined_mask.astype(np.float32),
            "nascent_mask": nascent_mask.astype(np.float32),
            "standard_mask": standard_mask.astype(np.float32),
            "probs": current_probs,
            "rate_of_rise": rate_of_rise,
            "regime": regime,
            "time_delta_mins": time_delta_mins
        }


#
   ================================================================

# 5. DISPATCHER: GPS EXTRACTION & ALERT GENERATION
#
   ================================================================
```

```
def dispatch_incident_alerts(analysis, raw_3ch, timestamp):
    combined_mask = analysis["combined_mask"]
    nascent_mask = analysis["nascent_mask"]
    rate_of_rise = analysis["rate_of_rise"]
    probs = analysis["probs"]
    regime = analysis["regime"]

    labeled_array, num_features = label(combined_mask)
    if num_features == 0:
        print(f"[{timestamp.strftime('%Y-%m-%d %H:%M:%S')}]
           Regime: {regime:5s} | Status: All Clear (0 Active
           Fronts)")
        return

    incidents = []
    print(f"\n" + "!" * 80)
    print(f"FIRE EMERGENCY ALERT DISPATCH: {timestamp.strftime('%
       Y-%m-%d %H:%M:%S')} UTC")
    print(f"Operational Regime: {regime} | Total Discrete Fronts:
        {num_features}")
    print("!" * 80)

    for cluster_id in range(1, num_features + 1):
        ys, xs = np.where(labeled_array == cluster_id)
        cluster_pixels = len(ys)
        area_km2 = cluster_pixels * PIXEL_AREA_KM2

        # Geographic coordinates of cluster centroid
        center_y = float(np.mean(ys))
        center_x = float(np.mean(xs))
        lat = LAT_MAX - (center_y / GRID_H) * (LAT_MAX - LAT_MIN)
        lon = LON_MIN + (center_x / GRID_W) * (LON_MAX - LON_MIN)

        # Thermal characteristics
        max_t39 = float(np.max(raw_3ch[0][ys, xs]))
        max_delta_t = float(np.max(raw_3ch[2][ys, xs]))
        max_rise = float(np.max(rate_of_rise[ys, xs]))
        mean_conf = float(np.mean(probs[ys, xs]))
        is_nascent = bool(np.any(nascent_mask[ys, xs]))

        # Alert classification
        if is_nascent and cluster_pixels <= 3:
            classification = "NASCENT_IGNITION_CRITICAL"
            action = "IMMEDIATE EVACUATION & AIR TANKER DISPATCH"
        elif cluster_pixels > 6:
            classification = "MAJOR_EXPANDING_WILDFIRE"
            action = "REGIONAL PERIMETER CONTAINMENT"
        else:
            classification = "ACTIVE_FLAMING_FRONT"
            action = "GROUND UNIT INTERVENTION"
```

```
        incident_record = {
            "incident_id": f"INC_{timestamp.strftime('%Y%m%d_%H%M
                ')}_{cluster_id:02d}",
            "timestamp": timestamp.isoformat(),
            "regime": regime,
            "classification": classification,
            "recommended_action": action,
            "latitude": round(lat, 4),
            "longitude": round(lon, 4),
            "estimated_area_km2": round(area_km2, 1),
            "pixel_count": cluster_pixels,
            "mean_confidence": round(mean_conf, 3),
            "peak_brightness_temp_39_K": round(max_t39, 1),
            "peak_delta_t_K": round(max_delta_t, 1),
            "rate_of_rise_K_per_5min": round(max_rise, 2)
        }
        incidents.append(incident_record)

        print(f" > FRONT #{cluster_id:02d} [{classification}]:")
        print(f"   Coordinates : Lat {lat:.4f} N , Lon {lon:.4f}
            E ")
        print(f"   Front Size  : ~{area_km2:.1f} km  ({
            cluster_pixels} px) | Peak T3.9: {max_t39:.1f} K")
        print(f"   Rate of Rise: +{max_rise:.2f} K / 5 min |
            Confidence: {mean_conf*100:.1f}%")
        print(f"   Directive   : {action}\n")

    # Persist structured incident log
    with open(ALERT_LOG_FILE, "a") as f_out:
        for inc in incidents:
            f_out.write(json.dumps(inc) + "\n")

    # Render visual diagnostic plot
    render_alert_snapshot(raw_3ch, analysis, incidents, timestamp
        )


def render_alert_snapshot(raw_3ch, analysis, incidents, timestamp
    ):
    combined_mask = analysis["combined_mask"]
    rate_of_rise = analysis["rate_of_rise"]
    date_tag = timestamp.strftime("%Y%m%d_%H%M%S")
    out_png = os.path.join(ALERT_DIR, f"incident_{date_tag}.png")

    fig, axes = plt.subplots(1, 3, figsize=(20, 5))

    # Panel 1: Physical Infrared Delta T Channel
    im0 = axes[0].imshow(raw_3ch[2] * LAND_MASK, cmap="inferno",
        origin="upper")
    axes[0].set_title(f"Thermal Contrast: Delta T [{timestamp.
        strftime('%H:%M:%S')} UTC]")
```

```
    fig.colorbar(im0, ax=axes[0], label="Kelvin")

    # Panel 2: Temporal Rate-of-Rise Derivative
    im1 = axes[1].imshow(rate_of_rise * LAND_MASK, cmap="coolwarm
       ", vmin=-2.0, vmax=5.0, origin="upper")
    axes[1].set_title("Temporal Rate-of-Rise: d(Delta T)/dt (K /
       5 min)")
    fig.colorbar(im1, ax=axes[1], label="d(Kelvin)/dt")

    # Panel 3: Actionable Alert Overlay
    axes[2].imshow(raw_3ch[2] * LAND_MASK, cmap="gray", origin="
       upper")

    # Plot standard active fronts
    std_y, std_x = np.where((analysis["standard_mask"] == 1.0) &
       (analysis["nascent_mask"] == 0.0))
    if len(std_y) > 0:
        axes[2].scatter(std_x, std_y, color="orange", s=30, label
           ="Active Fire Front")

    # Highlight nascent early ignitions in bright red
    nas_y, nas_x = np.where(analysis["nascent_mask"] == 1.0)
    if len(nas_y) > 0:
        axes[2].scatter(nas_x, nas_y, color="red", s=70, marker="
           *", label="Incipient Ignition (Nascent)")

    for inc in incidents:
        c_x = int((inc["longitude"] - LON_MIN) / (LON_MAX -
           LON_MIN) * (GRID_W - 1))
        c_y = int((LAT_MAX - inc["latitude"]) / (LAT_MAX -
           LAT_MIN) * (GRID_H - 1))
        axes[2].text(c_x + 3, c_y - 3, inc["incident_id"][-5:],
           color="cyan", fontsize=9, weight="bold")

    axes[2].set_title(f"Civil Protection Map ({len(incidents)}
       Incidents Detected)")
    axes[2].legend(loc="lower left")

    plt.tight_layout()
    plt.savefig(out_png, dpi=200)
    plt.close()
    print(f"Incident visualization saved to: {out_png}")


#
   ================================================================

# 6. RUNTIME DISPATCH MODES
#
   ================================================================
```

```
def run_simulation_stream():
    """
    Sequential simulation across historical unseen scenes to test
        temporal tracking.
    """
    print(f"Reading sequence records from: {INDEX_PATH}")
    df_index = pd.read_csv(INDEX_PATH)
    # Sort chronologically to preserve physical temporal flow
    df_seq = df_index.sort_values(by="timestamp").reset_index(
       drop=True)

    # Isolate test split scenes with ground truth fires
    test_fire_scenes = df_seq[(df_seq["split"] == "test") & (
       df_seq["fire_pixels"] >= 3)].copy()
    if len(test_fire_scenes) == 0:
        test_fire_scenes = df_seq[df_seq["split"] == "test"].copy
           ()

    print(f"Loaded {len(test_fire_scenes)} sequential test frames
        for simulation.")
    tracker = TemporalFireTracker()

    for idx, row in test_fire_scenes.iterrows():
        sample_path = row["file_path"]
        dt = pd.to_datetime(row["timestamp"])

        if not os.path.exists(sample_path):
            continue

        data = np.load(sample_path)
        raw_3ch = data["image"]  # [3, 120, 250]

        analysis = tracker.analyze_frame(raw_3ch, dt)
        dispatch_incident_alerts(analysis, raw_3ch, dt)
        time.sleep(0.5)  # Simulated stream cadence


def run_live_daemon():
    """
    Continuous real-time poller interfacing with the EUMETSAT
       Data Store API.
    """
    import eumdac
    from satpy import Scene
    from pyresample.geometry import AreaDefinition

    area_def = AreaDefinition(
        area_id="northern_algeria",
        description="Northern Algeria Target Domain",
        proj_id="latlong",
        projection={"proj": "latlong", "datum": "WGS84"},
```

```
        width=GRID_W,
        height=GRID_H,
        area_extent=[LON_MIN, LAT_MIN, LON_MAX, LAT_MAX]
    )

    print("Connecting to EUMETSAT Data Store API...")
    token = eumdac.AccessToken((CONSUMER_KEY, CONSUMER_SECRET))
    datastore = eumdac.DataStore(token)
    collection = datastore.get_collection("EO:EUM:DAT:MSG:MSG15-
       RSS")
    print("Connected. Listening for 5-minute Rapid Scan
       observations...\n")

    tracker = TemporalFireTracker()
    last_processed_time = None

    while True:
        try:
            now_utc = datetime.utcnow()
            # Search last 15 minutes
            start_window = now_utc - timedelta(minutes=15)
            products = collection.search(dtstart=start_window,
               dtend=now_utc)

            latest_product = None
            for prod in products:
                latest_product = prod
                break

            if latest_product is None:
                print(f"[{datetime.now().strftime('%H:%M:%S')}]
                   Polling... Waiting for new RSS product.")
                time.sleep(60)
                continue

            prod_dt = latest_product.sensing_start
            if last_processed_time is not None and prod_dt <=
               last_processed_time:
                time.sleep(60)
                continue

            print(f"\nNew Satellite Frame Acquired: {prod_dt.
               strftime('%Y-%m-%d %H:%M:%S')} UTC")

            # Stream directly into memory via temporary directory
            with tempfile.TemporaryDirectory() as temp_dir:
                with latest_product.open() as f_src:
                    temp_archive = os.path.join(temp_dir, "
                       incoming.zip")
                    with open(temp_archive, "wb") as f_dst:
                        shutil.copyfileobj(f_src, f_dst)
```

```
                # Extract .nat
                target_nat = None
                if zipfile.is_zipfile(temp_archive):
                    with zipfile.ZipFile(temp_archive, "r") as z:
                        for n in z.namelist():
                            if n.endswith(".nat") or "MSG" in n:
                                z.extract(n, temp_dir)
                                target_nat = os.path.join(
                                    temp_dir, n)
                                break
                else:
                    target_nat = temp_archive

                scn = Scene(filenames=[target_nat], reader="
                    seviri_l1b_native")
                scn.load(["IR_039", "IR_108"])
                resampled = scn.resample(area_def)

                t39 = np.nan_to_num(resampled["IR_039"].values.
                    astype(np.float32), nan=0.0)
                t108 = np.nan_to_num(resampled["IR_108"].values.
                    astype(np.float32), nan=0.0)
                delta_t = t39 - t108
                raw_3ch = np.stack([t39, t108, delta_t], axis=0)

            # Analyze frame through temporal pipeline
            analysis = tracker.analyze_frame(raw_3ch, prod_dt)
            dispatch_incident_alerts(analysis, raw_3ch, prod_dt)
            last_processed_time = prod_dt

        except Exception as e:
            print(f"Daemon Warning: {e}. Retrying in 30 seconds
                ...")
            time.sleep(30)


#
    ================================================================

# 7. EXECUTION ENTRY POINT
#
    ================================================================

if __name__ == "__main__":
    if MONITORING_MODE == "SIMULATION":
        run_simulation_stream()
    elif MONITORING_MODE == "LIVE":
        run_live_daemon()
    else:
        sys.exit(f"Unknown MONITORING_MODE: '{MONITORING_MODE}'.
```

```
            Choose 'SIMULATION' or 'LIVE'.")
```

Step 2: Testing the Temporal Tracker Immediately

To test the system immediately without waiting for real-time EUMETSAT API polling:
Notice that `MONITORING_MODE = "SIMULATION"` is set by default.
Run the script directly from your terminal:

```
python live_fire_monitor.py
```

What You Will Observe During Execution

- **Temporal Differential Tracking:**
  - For the first frame (t_0), the system calculates baseline spatial probabilities.
  - On subsequent frames (t_1, t_2), it computes the normalized rate-of-rise $\frac{\partial(\Delta T)}{\partial t}$.
- **Early Incipient Ignition Tagging:**
  - Stable terrain and gradually warming rocks register near 0.0 K / 5 min.
  - Rapidly accelerating combustion points trigger $\geq +1.5$ K / 5 min jumps, generating the high-priority NASCENT_IGNITION_CRITICAL tag and an actionable civil protection directive.
- **Structured Alert Logs & Maps:**
  - Each alert appends a structured JSON record into `live_alerts/active_incidents.jsonl` containing coordinates, front size in km^2, peak temperatures, and confidence metrics.
  - Whenever active fronts are identified, a visual alert report with star markers for nascent ignitions is saved into `live_alerts/incident_YYYYMMDD_HHMMSS.png`.

Run the script and inspect the terminal alerts.